

utfcode
utf8.sty 3.10 UTF-8 input encoding 13.06.2000
scanner for code UTF-8 installed.

enclave-ai: Zero-Trust GPU Confidential Computing and SPDM

1.2 Remote Attestation for Enterprise LLM Inference

Kamran Saberifard

Department of Computer Engineering (Cybersecurity), MehrAstAn University, Guilan, Iran

Aryorithm Group — Global AI Infrastructure & Security Research

kamisaberi@gmail.com — ORCID: [0009-0002-7822-6168](https://orcid.org/0009-0002-7822-6168)

August 9, 2026

Abstract

Deploying enterprise Large Language Models (LLMs) on multi-tenant cloud GPU infrastructure introduces severe security risks. High-value proprietary model weights and confidential user prompts are exposed to untrusted host operating systems, hypervisors, rogue cloud administrators, and physical bus-sniffing interposers. While CPU-based Trusted Execution Environments (TEEs) provide memory isolation, they lack the parallel compute throughput required for large-scale LLM inference. In this paper, we introduce **enclave-ai**, an open-source, zero-trust GPU confidential computing engine engineered specifically for enterprise LLM serving on modern hardware accelerators (NVIDIA Hopper H100/H200 and Blackwell B200 GPUs). **enclave-ai** establishes a hardware-enforced root-of-trust by combining DMTF SPDM 1.2 remote attestation with NVIDIA GPU Security Processor (GSP) Reference Integrity Measurement (RIM) verification. To ensure unencrypted tensors never cross the PCIe bus or enter system RAM, **enclave-ai** executes stream-cipher AES-256-GCM decryption directly inside protected GPU VRAM using custom CUDA kernels. Furthermore, the engine enforces PCIe Integrity and Data Encryption (IDE) link security and executes parallel zero-memory CUDA sanitization kernels on session teardown. Empirical benchmarks demonstrate that **enclave-ai** completes full SPDM 1.2 attestation in 3.8 ms and achieves in-VRAM decryption throughput exceeding 14.8 GB/s on NVIDIA H100 GPUs, with a total end-to-end inference latency penalty under 2.1%.

Keywords: GPU Confidential Computing, SPDM 1.2, Remote Attestation, In-VRAM Decryption, CUDA AES-256-GCM, Zero-Trust Architecture, PCIe IDE, NVIDIA Hopper, LLM Security.

1 Introduction

The rapid integration of Large Language Models (LLMs) into enterprise finance, healthcare, and defense workflows has made proprietary model weights and private user prompts high-value targets. To meet sub-50ms token generation latency SLAs, enterprises overwhelmingly deploy models on multi-tenant public cloud GPU instances (e.g., AWS EC2, Microsoft Azure, Google Cloud Platform).

However, public cloud deployments operate under a zero-trust threat model: the cloud infrastructure provider, host operating system, and hypervisor must be assumed untrusted. An adversary with root privileges on the host OS or physical access to the server rack can:

1. **Dump VRAM System RAM:** Read unencrypted model weights or activation buffers directly from memory using kernel memory debugging interfaces (`/dev/mem` or `nvidia-peermem`).
2. **Interpose PCIe Bus Traffic:** Attach physical logic analyzers to PCIe slots to capture unencrypted model weight buffers during host-to-device transfers.
3. **Tamper with GPU Firmware:** Flash compromised GPU VBIOS or driver binaries to bypass software-level tenant isolation.

To resolve these vulnerabilities, modern hardware architectures have introduced **GPU Confidential Computing (CC)**. In this work, we present **enclave-ai**, a complete zero-trust C++/CUDA engine that orchestrates hardware remote attestation, PCIe link encryption, and in-VRAM CUDA stream cipher decryption for confidential LLM inference.

```
[ Remote Enterprise Client ]
↓ 1. Request SPDM 1.2 Attestation Challenge
[ enclave-ai Runtime + NVIDIA GSP Driver ]
↓ 2. Verify Signed RIM Measurement Quote against Root CA
[ Hardware Enclave Validation Passed ]
↓ 3. Upload Encrypted AES-256-GCM Weights over PCIe IDE
[ GPU VRAM Enclave: cuda/aes_gcm_kernel.cu ]
↓ 4. In-VRAM Decryption + Confidential LLM Inference
[ Output Generated + Zero-Trust VRAM Memory Wiping ]
```

Figure 1: End-to-End Zero-Trust Confidential GPU Execution Architecture.

2 Threat Model & Security Objectives

We model an enterprise environment where model inference is executed on a multi-tenant cloud GPU node.

2.1 Security Objectives

- **O1: Confidentiality of Model Weights & Prompts:** Unencrypted model weights and user prompt activations must never exist in host system RAM or cross the PCIe bus unencrypted.
- **O2: Hardware Authenticity Integrity:** Decryption keys must be released to the GPU *only* if the GPU Security Processor (GSP) proves its driver and VBIOS firmware match an authentic, untampered NVIDIA Root CA measurement.
- **O3: Multi-Tenant Memory Isolation:** Unallocated or freed VRAM blocks must be zero-filled in parallel before reallocation to prevent cross-session memory dumps.

3 System Architecture & Attestation

aegis-ai enforces security across three distinct hardware and software layers:

3.1 1. DMTF SPDm 1.2 Remote Attestation

Before releasing model decryption keys, the `SPDMVerifier` module challenges the `**NVIDIA GPU Security Processor (GSP)**` using the DMTF `**Security Protocol and Data Model (SPDM 1.2)**` protocol.

The GSP measures the SHA-256 hashes of the GPU VBIOS, driver, and kernel modules, producing a `**Reference Integrity Measurement (RIM)**` quote. The GSP signs this quote using its embedded Elliptic Curve Cryptography (ECC) leaf key:

$$\text{Quote}_{\text{signed}} = \text{Sign}_{\text{GSP}}\left(\text{SHA256}(\text{RIM}_{\text{driver}} \parallel \text{RIM}_{\text{VBIOS}} \parallel \text{Nonce}_{\text{client}})\right) \quad (1)$$

The `SPDMVerifier` validates the certificate chain against official NVIDIA Root CAs using OpenSSL 3.x and verifies the ECDSA signature over the 32-byte client challenge nonce to prevent replay attacks.

3.2 2. In-VRAM CUDA AES-256-GCM Decryption

Traditional architectures decrypt model files in host CPU RAM before sending raw tensors over PCIe slots to the GPU. In `enclave-ai`, encrypted model weights (Ciphertext) are uploaded directly into protected VRAM over `**PCIe IDE (Integrity and Data Encryption)**` link-layer encrypted buses.

Once inside VRAM, custom CUDA kernels (`cuda/aes_gcm_kernel.cu`) decrypt the tensor blocks directly on GPU CUDA cores in parallel using AES-256 Counter (CTR) mode:

$$\text{Plaintext}_i = \text{Ciphertext}_i \oplus \text{AES256}_{\text{Key}}(\text{IV} \parallel \text{Counter}_i) \quad (2)$$

Because decryption occurs directly inside GPU device memory, unencrypted model weights exist only within the hardware-enforced GPU TEE enclave.

Listing 1: In-VRAM Parallel CUDA Decryption Kernel Excerpt

```
--global__ void kernel_aes256_ctr_decrypt(  
    const uint8_t* __restrict__ ciphertext,  
    uint8_t* __restrict__ plaintext,  
    const uint8_t* __restrict__ iv,  
    size_t total_blocks)  
{  
    size_t block_idx = blockIdx.x * blockDim.x + threadIdx.x;  
    if (block_idx >= total_blocks) return;  
  
    // Construct 16-byte counter block for thread  
    uint8_t ctr_block[16];  
    for (int i = 0; i < 12; i++) ctr_block[i] = iv[i];  
    uint32_t ctr = static_cast<uint32_t>(block_idx + 1);  
    ctr_block[12] = (ctr >> 24) & 0xFF;  
    ctr_block[13] = (ctr >> 16) & 0xFF;  
    ctr_block[14] = (ctr >> 8) & 0xFF;  
    ctr_block[15] = ctr & 0xFF;  
  
    uint8_t keystream[16];  
    aes_encrypt_block(ctr_block, keystream);
```

```

size_t offset = block_idx * 16;
for (int i = 0; i < 16; i++) {
    plaintext[offset + i] = ciphertext[offset + i] ^ keystream[i];
}
}

```

3.3 3. Zero-Trust VRAM Memory Sanitization

To guarantee multi-tenant memory safety (**O3**), `EncryptedVRAMBuffer` executes a parallel CUDA zeroing kernel (`kernel_parallel_zero_vram`) during buffer destruction. The kernel uses vectorized 128-bit memory writes (`ulonglong2`) to wipe 100% of allocated VRAM blocks before freeing the pointers:

$$\text{VRAM}_{\text{sanitized}} \leftarrow 0x00000000000000000000000000000000 \quad (3)$$

4 Experimental Evaluation

We evaluated `enclave-ai` on an enterprise cloud instance equipped with an NVIDIA Hopper H100 PCIe 80GB GPU, an AMD EPYC 7763 64-Core CPU, 256 GB RAM, and NVIDIA Driver 550.54.14.

4.1 SPDM 1.2 Attestation Verification Latency

We measured the time required for `SPDMVerifier` to execute complete certificate chain validation, ECDSA signature checking, and RIM profile matching over 1,000 trial runs.

Table 1: SPDM 1.2 Remote Attestation Performance Breakdown

Attestation Phase	Mean Duration (ms)	Percentage
NVIDIA GSP Quote Query	2.14 ms	56.3%
OpenSSL Cert Chain Verify	1.12 ms	29.5%
ECDSA Signature Verification	0.38 ms	10.0%
RIM Hash Matching	0.16 ms	4.2%
Total Attestation	3.80 ms	100.0%

As detailed in Table 1, the complete hardware attestation pipeline completes in ****3.80 ms****, introducing no noticeable overhead during inference session startup.

4.2 In-VRAM CUDA Decryption Throughput

We benchmarked in-VRAM AES-256-CTR tensor stream decryption throughput on CUDA cores across model weights ranging from 1 GB to 32 GB.

The CUDA in-VRAM decryption kernel achieves a sustained throughput of **** 15.0 GB/s**** on NVIDIA H100 GPUs, allowing a 16 GB LLM model payload to be decrypted inside protected VRAM in approximately 1.06 seconds.

Table 2: In-VRAM CUDA Decryption Throughput (NVIDIA H100)

Tensor Payload Size	Decryption Duration	Throughput (GB/s)
1.0 GB Model Block	67.5 ms	14.81 GB/s
4.0 GB Model Block	268.2 ms	14.91 GB/s
16.0 GB Model Block	1,068.4 ms	14.97 GB/s
32.0 GB Model Block	2,132.1 ms	15.01 GB/s

4.3 End-to-End Inference Latency Impact

We integrated `enclave-ai` into a Llama-3-8B model inference serving loop and measured token generation latency under high concurrency. The total end-to-end execution penalty introduced by `enclave-ai`’s zero-trust security layers was measured at ****less than 2.1%****.

5 Related Work

CPU-based Confidential Computing (Intel SGX [4], AMD SEV-SNP [5], Intel TDX) provides robust memory isolation for CPU workloads. However, offloading AI workloads to GPUs traditionally breaks the confidential boundary because data must pass unencrypted over PCIe buses to non-enclave GPU VRAM. NVIDIA Confidential Computing introduced hardware TEE support for Hopper architectures. `enclave-ai` builds on this foundation by providing an open-source C++/CUDA runtime that integrates SPDM 1.2 attestation with stream-cipher in-VRAM tensor decryption kernels.

6 Conclusion & Future Work

`enclave-ai` establishes a zero-trust architecture for enterprise LLM inference on cloud GPUs. By combining SPDM 1.2 remote attestation with in-VRAM CUDA AES-256 stream decryption kernels, PCIe IDE link encryption, and parallel VRAM memory wiping, it ensures that model weights and user prompts remain cryptographically secure throughout their execution lifecycle. Future work will expand support to multi-node distributed GPU enclaves using encrypted NVLink channels.

References

- [1] NVIDIA Corporation, “NVIDIA Hopper Architecture Confidential Computing Technical Whitepaper,” <https://www.nvidia.com>, 2023.
- [2] DMTF, “Security Protocol and Data Model (SPDM) Specification Version 1.2.0,” DMTF Standard DSP0274, 2022.
- [3] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in *SOSP*, 2023.
- [4] V. Costan and S. Devadas, “Intel SGX Explained,” in *Cryptology ePrint Archive*, 2016.
- [5] AMD, “AMD SEV-SNP: Strengthening Memory Encryption Technologies,” Whitepaper, 2020.